

Week 2 — AI Chatbot Widget Requirements Review

TaxNexus (soloai_starter) · branch `feat/ai-chatbot-widget` (PR #1) · Anthropic Claude Sonnet 4.6 · Reviewed 2026-07-03

Overall: strong pass. All six sections functionally meet the requirements. A few of the checklist's *stated mechanisms* differ from the implementation (all intentional and justified), plus two genuine minor gaps (error-state retry, mid-stream error visibility) — both since fixed in PR #1 (see addendum).

1) Pass/Fail Checklist

1) Chatbot Widget (UI + Behavior) — PASS (1 partial)

Item	Verdict	Evidence
Lightweight embeddable component	PASS	<code>Chatbot.svelte</code> , lazy-imported
Launcher opens/closes reliably	PASS	<code>toggle()</code> , close button, <code>Escape</code> handler
Launcher + header + scrollable pane + input/send	PASS	launcher, navy header, <code>overflow-y-auto</code> pane, composer
Sending/loading state	PASS	<code>status='streaming'</code> , send spinner, input disabled
Streaming/typing indicator	PASS	<code>loading</code> <code>loading-dots</code> in empty assistant bubble
Error state with retry option	PARTIAL	Error alert + Dismiss/Snooze; no explicit Retry (now fixed)

2) Input Handling & Security — PASS

Item	Verdict	Evidence
No <code>{@html}</code> for user content	PASS	renders <code>{message.content}</code> (auto-escaped)
Input trimmed, empty blocked client-side	PASS	<code>send()</code> trims + early-returns; send disabled when empty
Server Zod validation (length, role, content)	PASS	role enum, <code>content</code> <code>≤4000</code> , <code>messages</code> <code>1–20</code>
XSS / injection risks	LOW	no <code>{@html}</code> / <code>eval</code> ; content-type gated; client can't inject <code>system</code> role

3) AI Integration (Server-Side Proxy) — PASS (2 mechanism deviations)

Item	Verdict	Evidence
Claude calls server-side via <code>+server.ts</code>	PASS	<code>client.messages.stream(...)</code> in the route
Key not exposed to client	PASS	<code>getEnv().ANTHROPIC_API_KEY</code> in <code>\$lib/server/env.ts</code> (server-only). Deviation: <code>process.env</code> , not <code>\$env/static/private</code>
Env validation for the key	DEVIATION	<code>parseAnthropicKey()</code> enforces <code>sk-ant-</code> prefix in <code>env.ts</code> . Hand-rolled (house style), not Zod / not <code>env-validation.ts</code>

4) Conversation State — PASS (filename deviation)

Item	Verdict	Evidence
Store tracks history + flags	PASS	<code>\$lib/stores/chat.svelte.ts</code> . Deviation: <code>.svelte.ts</code> (mandatory for <code>\$state</code> runes), not <code>.ts</code>
Sliding window last 10 → API	PASS	<code>chatStore.getRecent(10)</code>
sessionStorage respects session boundaries	PASS	<code>syncUser()</code> owner-tags + clears on guest↔user / A↔B

5) Reliability, Rate Limits, Production Safety — PASS

Item	Verdict	Evidence
AbortController 30s	PASS	<code>REQUEST_TIMEOUT_MS = 30_000</code> → <code>controller.abort()</code>
Non-200 upstream handled (429/5xx)	PASS	pre-flight try/catch → 502/504; upstream 429 → <code>upstream_busy</code>
Clear error payload to client	PASS	<code>{error, requestId}</code> on 4xx/5xx; 429 friendly
Rate limiting + UI 429 handling	PASS	20/60s server; client → <code>rate-limited</code> friendly copy
Correlation IDs every request	PASS	<code>randomUUID()</code> logged + <code>X-Request-Id</code> on all responses
Content-free logging	PASS	logs only id, userId, messageCount, latency, tokens, kind

6) App Embedding & Theming — PASS

Item	Verdict	Evidence
Embedded in <code>+layout.svelte</code>	PASS	rendered after <code><Footer></code>
Lazy-loaded via <code>onMount</code> dynamic import	PASS	<code>+layout.svelte:34–37</code>
Inherits DaisyUI + CSS variables	PASS	DaisyUI classes; brand via inline hex (note: not app CSS vars)
Brand palette navy <code>#0D1B2A</code> / amber <code>#F0A500</code>	PASS	<code>const NAVY</code> , <code>const AMBER</code>
No global CSS conflicts	PASS	utility + inline styles, fixed <code>z-50</code> ; no global selectors
Guests see sign-in prompt	PASS	<code>{#if !authenticated}</code> banner
Trial/paid get full chat	PASS	authenticated → full chat + tier badge (auth-aware)

2) Missing / Deviating Requirements (with file paths)

1. **No explicit Retry button** on the error state — `src/lib/components/Chatbot.svelte` . *Genuine gap*.
2. **Mid-stream failures masqueraded as success** — `src/routes/api/chat/+server.ts` . *Genuine gap* (pre-stream errors were already handled).
3. **Key access mechanism** — `src/lib/server/env.ts` uses `process.env` via `getEnv()` , not `$env/static/private` . *Deviation only* (still server-only).
4. **Validation style** — `env.ts` hand-rolled, not Zod / not `env-validation.ts` . *Deviation only* (house style).
5. **Store filename** — `chat.svelte.ts` , not `chat.ts` . *Required deviation* (Svelte 5 runes).

3) Top 3 Recommendations

1. **Add a Retry affordance to the error state** — re-send the last failed user message from the error alert. The only checklist item that was an actual UX gap.
2. **Make mid-stream failures visible to the client** — mark the assistant message as `error` on abnormal stream close (in-band sentinel), so a dropped stream doesn't read as a normal answer.
3. **Sharpen error observability** — return `X-Request-Id` on *all* client-facing error responses and distinguish upstream Anthropic `429` ("service busy") from `5xx`.

4) Concrete Next Steps

- `Chatbot.svelte` : add `retry()` that resets the last errored placeholder and re-invokes the completion; render a **Retry** button beside Snooze/Dismiss.
- `+server.ts` : emit an in-band sentinel on the stream's error path; client detects it → mark error + retry state. Add `X-Request-Id` to all error responses.
- Optional (grading nicety): note the three intentional deviations in the submission — the store must stay `.svelte.ts`; the key is server-only via `$lib/server/env.ts`; validation follows the repo's hand-rolled `env.ts` pattern.

Status update — all three recommendations implemented (PR #1, commit `d410691`): Retry button (`data-testid=chat-retry`); null-byte mid-stream error sentinel that the client strips and surfaces as an error/retry state; and `X-Request-Id` + `requestId` on every client-facing response, with upstream `429` surfaced distinctly as `upstream_busy`. Verified: `svelte-check` 0/0, build passes, live 400/415 carry the id + payload, happy path streams.

Files reviewed: `src/routes/api/chat/+server.ts`, `src/lib/stores/chat.svelte.ts`, `src/lib/components/Chatbot.svelte`, `src/routes/+layout.svelte`, `src/lib/server/env.ts`. Generated with Claude Code.